MDTS4214_737_6

SULAGNA ROY

2026-03-20

Problem Set 7: Shrinkage Methods

Comparison of least squares, ridge regression, LASSO and Elastic Net

You would be working with Hitters data set available in the ISLR library. You have to predict the Salary based on all available variables. First make sure that you have deleted all rows with one or more missing values.

```
library(ISLR)
## ??Hitters
original_data=Hitters
```

Information on the data set:

A data frame with 322 observations of major league players on the following 20 variables.

*(i)*AtBat:Number of times at bat in 1986

*(ii)*Hits:Number of hits in 1986

*(iii)*HmRun:Number of home runs in 1986

*(iv)*Runs:Number of runs in 1986

*(v)*RBI:Number of runs batted in in 1986

*(vi)*Walks:Number of walks in 1986

*(vii)*Years:Number of years in the major leagues

*(viii)*CAtBat:Number of times at bat during his career

*(ix)*CHits:Number of hits during his career

*(x)*CHmRun:Number of home runs during his career

*(xi)*CRuns:Number of runs during his career

*(xii)*CRBI:Number of runs batted in during his career

*(xiii)*CWalks:Number of walks during his career

*(xiv)*League:A factor with levels A and N indicating player's league at the end of 1986

*(xv)*Division:A factor with levels E and W indicating player's division at the end of 1986

*(xvi)*PutOuts:Number of put outs in 1986

*(xvii)*Assists:Number of assists in 1986

*(xviii)*Errors:Number of errors in 1986

*(xix)*Salary:1987 annual salary on opening day in thousands of dollars

*(xx)*NewLeague:A factor with levels A and N indicating player's league at the beginning of 1987

Here, 'Salary' will be our Response variable.

```
cleaned_data=na.omit(original_data)
nrow_cleaned_data=dim(cleaned_data)[1][[1]];nrow_cleaned_data
```

```
## [1] 263
```

1. Split the data set into a training set and a test set randomly keeping 100 observations in test set and remaining in the training set.

```
# Ids of full data size
id=1:263

# 80% sample on ids of original data size for training sample
set.seed(123)
sample_id_new=sample(id,100)

# Training data
test_data_new=cleaned_data[sample_id_new,]

#Test data
train_data_new=cleaned_data[-sample_id_new,]
```

2. Train the model using least squares and report the train MSE and test MSE.

Fitted Least Square Model:

```
#Fitted Least Square Model
OLS_model=lm(Salary~.,data=train_data_new);OLS_model
```

```
##
## Call:
## lm(formula = Salary ~ ., data = train_data_new)
##
## Coefficients:
## (Intercept)        AtBat         Hits        HmRun         Runs          RBI
##   212.67449     -2.04895      5.85495      5.43622     -2.49915      2.32756
##       Walks        Years       CAtBat        CHits       CHmRun        CRuns
##     6.40986     -8.30051      0.01569     -0.54632     -2.38189      1.56320
##        CRBI       CWalks      LeagueN    DivisionW      PutOuts      Assists
##     1.26993     -0.95946     44.32184   -149.54532      0.41589      0.43589
##      Errors   NewLeagueN
##    -5.50162    -15.79562
```

Training MSE:

```r
Salary_est_OLS_train=predict(OLS_model,data=train_data_new)
train_mse_OLS=mean((train_data_new$Salary-Salary_est_OLS_train)^2);train_mse_OLS
```

```
## [1] 105334.2
```

Test MSE:

```r
Salary_est_OLS_test=predict(OLS_model,test_data_new)
test_mse_OLS=mean((test_data_new$Salary-Salary_est_OLS_test)^2);test_mse_OLS
```

```
## [1] 87803.91
```

3. Fit ridge regression models on the training set, with grid values of lambda. Report the following:

```r
#Grid Values of lambda
lambda_grid=10^seq(5,-2,length=100)

#Fitting of Ridge Regression:
library("glmnet")
```

```
## Warning: package 'glmnet' was built under R version 4.5.3

## Loading required package: Matrix

## Warning: package 'Matrix' was built under R version 4.5.2

## Loaded glmnet 4.1-10
```

```r
X_TRAIN=model.matrix(Salary~.,data=train_data_new)

Y_TRAIN=train_data_new$Salary

X_TEST=model.matrix(Salary~.,data=test_data_new)
Y_TEST=test_data_new$Salary

hitters_ridge=glmnet(X_TRAIN,Y_TRAIN,alpha=0,lambda=lambda_grid);hitters_ridge
```

```
##
## Call:  glmnet(x = X_TRAIN, y = Y_TRAIN, alpha = 0, lambda = lambda_grid)
##
##     Df  %Dev Lambda
## 1   19  2.03 100000
## 2   19  2.38  84980
## 3   19  2.77  72210
## 4   19  3.23  61360
## 5   19  3.75  52140
## 6   19  4.35  44310
## 7   19  5.04  37650
## 8   19  5.82  31990
```

```
## 9   19  6.70  27190
## 10  19  7.68  23100
## 11  19  8.78  19630
## 12  19  9.99  16680
## 13  19 11.32  14170
## 14  19 12.76  12050
## 15  19 14.29  10240
## 16  19 15.92   8697
## 17  19 17.61   7391
## 18  19 19.35   6280
## 19  19 21.12   5337
## 20  19 22.89   4535
## 21  19 24.62   3854
## 22  19 26.30   3275
## 23  19 27.91   2783
## 24  19 29.42   2364
## 25  19 30.83   2009
## 26  19 32.13   1707
## 27  19 33.32   1451
## 28  19 34.39   1233
## 29  19 35.37   1048
## 30  19 36.25    890
## 31  19 37.05    756
## 32  19 37.77    643
## 33  19 38.42    546
## 34  19 39.01    464
## 35  19 39.55    394
## 36  19 40.04    335
## 37  19 40.50    285
## 38  19 40.92    242
## 39  19 41.31    206
## 40  19 41.69    175
## 41  19 42.04    148
## 42  19 42.39    126
## 43  19 42.73    107
## 44  19 43.06     91
## 45  19 43.38     77
## 46  19 43.71     66
## 47  19 44.03     56
## 48  19 44.34     48
## 49  19 44.65     40
## 50  19 44.95     34
## 51  19 45.24     29
## 52  19 45.51     25
## 53  19 45.77     21
## 54  19 46.01     18
## 55  19 46.23     15
## 56  19 46.43     13
## 57  19 46.60     11
## 58  19 46.76      9
## 59  19 46.90      8
## 60  19 47.02      7
## 61  19 47.12      6
## 62  19 47.22      5
```

```
## 63  19 47.29      4
## 64  19 47.36      4
## 65  19 47.42      3
## 66  19 47.47      3
## 67  19 47.51      2
## 68  19 47.56      2
## 69  19 47.58      2
## 70  19 47.62      1
## 71  19 47.63      1
## 72  19 47.66      1
## 73  19 47.67      1
## 74  19 47.70      1
## 75  19 47.70      1
## 76  19 47.73      0
## 77  19 47.73      0
## 78  19 47.75      0
## 79  19 47.76      0
## 80  19 47.77      0
## 81  19 47.78      0
## 82  19 47.78      0
## 83  19 47.79      0
## 84  19 47.79      0
## 85  19 47.79      0
## 86  19 47.80      0
## 87  19 47.80      0
## 88  19 47.80      0
## 89  19 47.80      0
## 90  19 47.81      0
## 91  19 47.81      0
## 92  19 47.81      0
## 93  19 47.81      0
## 94  19 47.81      0
## 95  19 47.81      0
## 96  19 47.81      0
## 97  19 47.81      0
## 98  19 47.81      0
## 99  19 47.81      0
## 100 19 47.81      0
```

(i) the dimension of the coefficient matrix so obtained.

```r
dim(coef(hitters_ridge))
```

```
## [1]  21 100
```

(ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of lambda. Comment on the output.

```r
# Regression Coefficients w.r.t. 50th & 60th choice of lambda
coef_50=coef(hitters_ridge)[,50];coef_50
```

```
##   (Intercept)   (Intercept)         AtBat          Hits         HmRun
```

```
##  106.30930133    0.00000000   -0.56861594    1.57781724    1.45165595
##          Runs           RBI         Walks         Years        CAtBat
##    0.26148608    2.54343128    2.88812942   -4.15498762    0.01212119
##         CHits        CHmRun         CRuns          CRBI        CWalks
##    0.12212743    0.02430066    0.22110562    0.24061114   -0.27212940
##       LeagueN     DivisionW       PutOuts       Assists        Errors
##   54.66429035 -145.12121490    0.35444338    0.26953832   -5.05469505
##    NewLeagueN
##  -14.05949208
```

```
coef_60=coef(hitters_ridge)[,60];coef_60
```

```
##   (Intercept)   (Intercept)         AtBat          Hits         HmRun
##  1.755634e+02  0.000000e+00 -1.408336e+00  3.329840e+00  1.626042e+00
##          Runs           RBI         Walks         Years        CAtBat
## -1.917524e-01  3.107071e+00  4.844059e+00 -7.920993e+00 -5.315477e-03
##         CHits        CHmRun         CRuns          CRBI        CWalks
##  1.602586e-01 -3.013027e-01  4.667053e-01  3.938615e-01 -6.029094e-01
##       LeagueN     DivisionW       PutOuts       Assists        Errors
##  5.686263e+01 -1.526506e+02  3.918256e-01  3.813593e-01 -5.711395e+00
##    NewLeagueN
## -2.698770e+01
```

```
# Corresponding l2 norms
l2_norm=function(beta){sqrt(sum(beta^2))}
l2_norm_50=l2_norm(coef_50);l2_norm_50
```

```
## [1] 188.708
```

```
l2_norm_60=l2_norm(coef_60);l2_norm_60
```

```
## [1] 241.3126
```

(iv) Test MSE corresponding to these two choices of lambda.

```
predict_y_50=predict(hitters_ridge,s=lambda_grid[50],newx=X_TEST)
predict_y_60=predict(hitters_ridge,s=lambda_grid[60],newx=X_TEST)

test_mse_50=mean((Y_TEST-predict_y_50)^2);test_mse_50
```

```
## [1] 89317.28
```

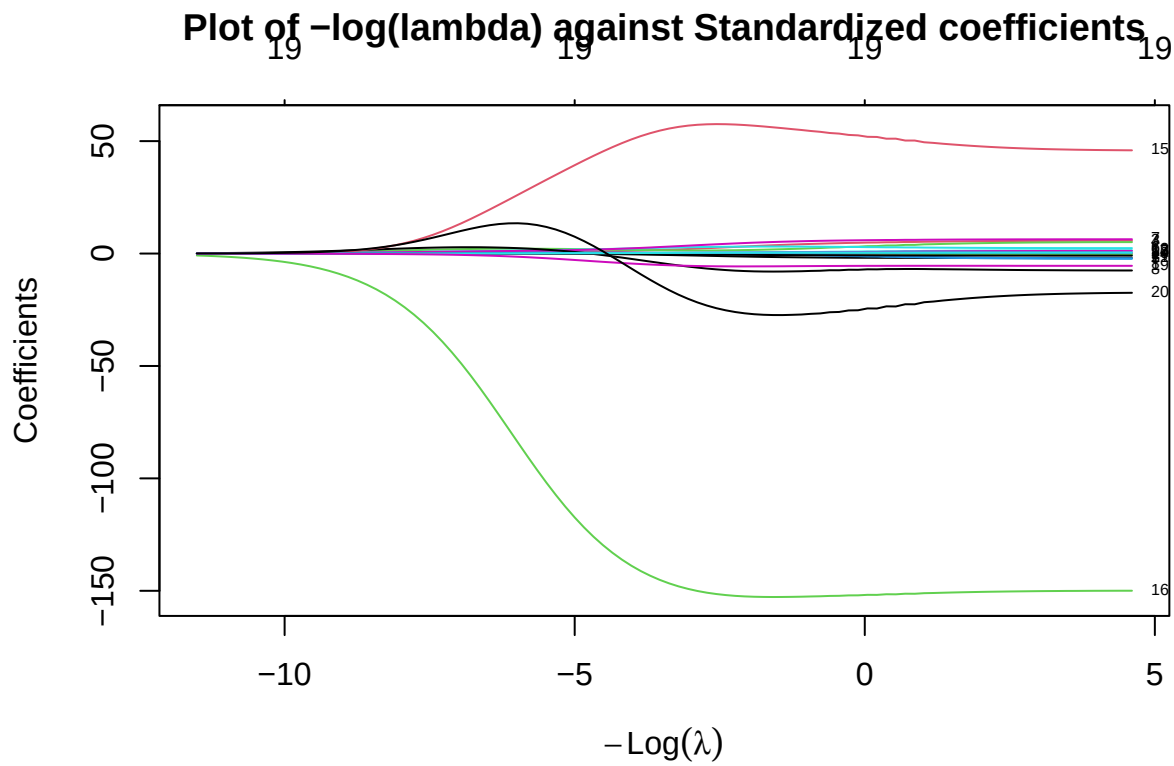
```
test_mse_60=mean((Y_TEST-predict_y_60)^2);test_mse_60
```

```
## [1] 87580.18
```

Comment: As the tuning parameter lambda falls the l2 norm rises and the test mse falls.

4. Make a line plot (for all the predictors) of the standardized coefficients against the grid value of 'lambda' chosen by you. Interpret the graph.

```
plot(hitters_ridge,xvar="lambda",label="TRUE",main="Plot of -log(lambda) against Standardized coefficie
```



Interpretation of the above Graph:

Sequentially predictors numbered 15,16,20 that are- **(i)** Predictor no. 15-Division:A factor with levels E and W indicating player's division at the end of 1986

(ii) Predictor no. 16-PutOuts:Number of put outs in 1986

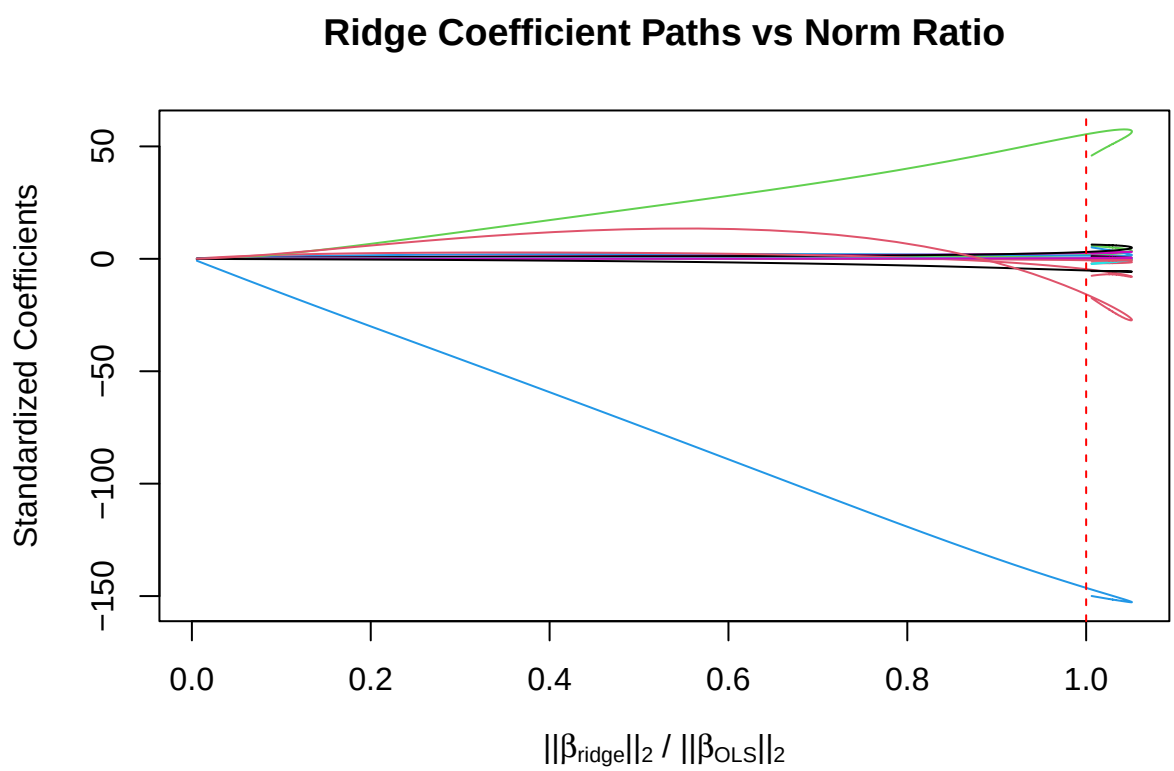
(iii) Predictor no. 20-NewLeague:A factor with levels A and N indicating player's league at the beginning of 1987

are most influential in predicting the response.

5. How would the graph in (4) change if instead of 'lambda' you plot on the x axis the ratio of l2 norm of the ridge regression coefficient to the least square regression coefficient.

```
ols_coef <- coef(OLS_model)[-1]   # remove intercept

# Compute L2 norm of OLS coefficients
ols_norm <- sqrt(sum(ols_coef^2))

# Extract ridge coefficients (excluding intercept)
ridge_coef <- as.matrix(hitters_ridge$beta)

# Compute L2 norms for each lambda
ridge_norms <- apply(ridge_coef, 2, function(b) sqrt(sum(b^2)))
```

```r
# Compute ratio
ratio <- ridge_norms / ols_norm

# Plot coefficient paths vs ratio
matplot(ratio, t(ridge_coef), type = "l", lty = 1,
        xlab = expression(paste("||", beta[ridge], "||"[2], " / ||", beta[OLS], "||"[2])),
        ylab = "Standardized Coefficients",
        main = "Ridge Coefficient Paths vs Norm Ratio")

# Add vertical reference line at ratio = 1 (OLS)
abline(v = 1, col = "red", lty = 2)
```



Comment:

When the x-axis is changed from lambda to ratio of l2 norm of the ridge regression coefficient to the least square regression coefficient, the plot becomes a reversed and rescaled version of the original coefficient path plot. The curves remain the same but are now parameterized by the degree of shrinkage, making interpretation more intuitive.

6.Run the Ridge regression on training data by choosing lambda by cross-validation. Find the test MSE for the optimal choice of lambda obtained using cross validation.

Applying Cross Validation:

```r
cv_ridge=cv.glmnet(X_TRAIN,Y_TRAIN,lambda=lambda_grid);cv_ridge
```

```
##
```

```
## Call:  cv.glmnet(x = X_TRAIN, y = Y_TRAIN, lambda = lambda_grid)
##
## Measure: Mean-Squared Error
##
##     Lambda Index Measure    SE Nonzero
## min  21.05    53  135980 33934       8
## 1se 107.23    43  164844 37061       4
```

Best Choice of lambda:

```
best_lambda <- cv_ridge$lambda.min
best_lambda
```

```
## [1] 21.04904
```

Predict on train set:

```
ridge_pred_train <- predict(cv_ridge, s = best_lambda, newx = X_TRAIN)
```

Predict on test set:

```
ridge_pred <- predict(cv_ridge, s = best_lambda, newx = X_TEST)
```

Computation of Test MSE:

```
test_mse_ridge <- mean((Y_TEST - ridge_pred)^2)
test_mse_ridge
```

```
## [1] 94635.78
```

7. Fit a LASSO model on the training set, with lambda chosen by cross-validation. Report the test MSE, along with the number of non-zero coefficient estimates.

LASSO with cross-validation:

```
cv_lasso <- cv.glmnet(X_TRAIN,Y_TRAIN, alpha = 1,lambda=lambda_grid)
```

Optimal lambda:

```
best_lambda_lasso <- cv_lasso$lambda.min
best_lambda_lasso
```

```
## [1] 21.04904
```

Predict on test set:

```
lasso_pred <- predict(cv_lasso, s = best_lambda_lasso, newx = X_TEST)
```

Test MSE:

```r
test_mse_lasso <- mean((Y_TEST - lasso_pred)^2)
test_mse_lasso
```

```
## [1] 94635.78
```

Extract coefficients:

```r
lasso_coef <- predict(cv_lasso, s = best_lambda, type = "coefficients")
```

Number of non-zero coefficients (excluding intercept):

```r
nonzero_coef <- sum(lasso_coef[-1] != 0)
nonzero_coef
```

```
## [1] 8
```

8. Fit an Elastic Net model on the training set, with lambda1 and lambda2 chosen by cross-validation. Report the test MSE. Also report the predictors with non-zero coefficients.

```r
# -------------------------------
# Step 1: Choose alpha (Elastic Net mixing)
# -------------------------------
alpha_values <- seq(0, 1, by = 0.1)

cv_errors <- rep(NA, length(alpha_values))

for(i in 1:length(alpha_values)){
  cv_model <- cv.glmnet(X_TRAIN, Y_TRAIN, alpha = alpha_values[i])
  cv_errors[i] <- min(cv_model$cvm)
}

# Best alpha
best_alpha <- alpha_values[which.min(cv_errors)]
best_alpha
```

```
## [1] 0
```

```r
# -------------------------------
# Step 2: Fit model with best alpha
# -------------------------------
cv_enet <- cv.glmnet(X_TRAIN, Y_TRAIN, alpha = best_alpha)

best_lambda <- cv_enet$lambda.min
best_lambda
```

```
## [1] 368.4021
```

```r
# -------------------------------
# Step 3: Predictions
# -------------------------------
enet_pred <- predict(cv_enet, s = best_lambda, newx = X_TEST)

# Test MSE
test_mse_enet <- mean((Y_TEST - enet_pred)^2)
test_mse_enet
```

```
## [1] 93231.02
```

```r
# -------------------------------
# Step 4: Coefficients
# -------------------------------
enet_coef <- predict(cv_enet, s = best_lambda, type = "coefficients")

# Non-zero coefficients
nonzero_index <- which(enet_coef[-1] != 0)
selected_variables <- rownames(enet_coef)[-1][nonzero_index]

selected_variables
```

```
##  [1] "AtBat"      "Hits"       "HmRun"      "Runs"       "RBI"
##  [6] "Walks"      "Years"      "CAtBat"     "CHits"      "CHmRun"
## [11] "CRuns"      "CRBI"       "CWalks"     "LeagueN"    "DivisionW"
## [16] "PutOuts"    "Assists"    "Errors"     "NewLeagueN"
```

9. Compare the test MSE 's of least squares regression, Ridge regression, LASSO and ELASTIC NET and comment on how accurately we can predict the Salary of a baseball player.

```r
Method=c("OLS","Ridge_Regression","LASSO","Elastic_Net")
Test_MSEs=c(test_mse_OLS,test_mse_ridge,test_mse_lasso,test_mse_enet)
Conclusion=data.frame(Method,Test_MSEs);Conclusion
```

```
##             Method Test_MSEs
## 1              OLS  87803.91
## 2 Ridge_Regression  94635.78
## 3            LASSO  94635.78
## 4      Elastic_Net  93231.02
```

Comment: Since the Ordinary Least Square Method produced the least Test MSE, we would like to use the predicted OLS model for maintaining highest accuracy in prediction of Salary of a Baseball player.